МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ

ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ

«НОВОСИБИРСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»

КАФЕДРА ТЕОРЕТИЧЕСКОЙ И ПРИКЛАДНОЙ ИНФОРМАТИКИ

ЛАБОРАТОРНАЯ РАБОТА

по дисциплине «Введение в искусственный интеллект и логическое программирование»

РЕКУРСИВНЫЕ СТРУКТУРЫ ДАННЫХ В ПРОЛОГЕ

НОВОСИБИРСК 2026

1. Цель работы

Приобретение навыков разработки программ на языке Visual Prolog для работы с рекурсивными структурами данных.

2. Задание

Написать программу, определяющую основные отношения на множествах:

а) ОБЪЕДИНЕНИЕ;

б) ПЕРЕСЕЧЕНИЕ;

в) ВЫЧИТАНИЕ;

г) ДЕКАРТОВО ПРОИЗВЕДЕНИЕ.

3. Описание разработанной программы

1) Структура базы знаний:
    - element = integer – тип элемента множества
    - set = element* – множество представлено как список целых чисел
    - pair = pair(element, element) – упорядоченная пара элементов
    - pair_set = pair* – множество пар
2) Вспомогательные предикаты:
    - member(X, List) – проверяет принадлежность элемента X списку List
    - append(List1, List2, Result) – сцепляет два списка в один
    - append_pairs(Pairs1, Pairs2, Result) – сцепляет два списка пар
    - add_unique(List, SetA, Current, Result) – добавляет в результат элементы из List, отсутствующие в SetA
3) Основные предикаты:
    - union(A, B, Result) – вычисляет объединение множеств A ∪ B (элементы, входящие хотя бы в одно из множеств)
    - intersection(A, B, Result) – вычисляет пересечение множеств A ∩ B (элементы, входящие в оба множества)
    - subtraction(A, B, Result) – вычисляет вычитание множеств A \ B (элементы из A, не входящие в B)
    - cartesian_product(A, B, Result) – вычисляет декартово произведение A × B (все упорядоченные пары (a, b), где a ∈ A, b ∈ B)
    - make_pairs(X, B, Pairs) – для элемента X строит пары со всеми элементами списка B

4) Принцип работы: при запуске программа задает два тестовых множества A и B в виде списков целых чисел, затем последовательно выполняет операции над множествами, используя рекурсивные предикаты для обхода списков и механизм возврата для проверки принадлежности элементов:

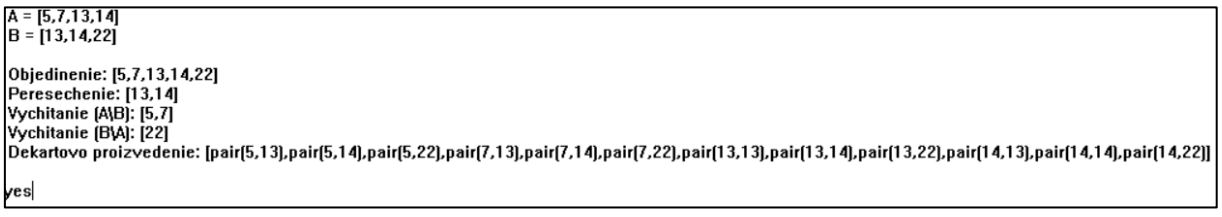
- Объединение – программа копирует все элементы первого множества, затем с помощью предиката add_unique добавляет элементы второго множества, которых нет в первом. Проверка наличия элемента осуществляется через предикат member, который рекурсивно просматривает список.

- Пересечение – рекурсивно обходит первое множество, для каждого элемента проверяет его принадлежность второму множеству, при успешной проверке элемент добавляется в результат. Ненужные альтернативы после нахождения элемента отбрасываются с помощью отсечения (!).

- Вычитание – рекурсивно обходит первое множество, включает в результат только те элементы, которые отсутствуют во втором множестве. Для предотвращения лишних возвратов используется отсечение (!).

- Декартово произведение – рекурсивно обходит первое множество, для каждого элемента вызывает предикат make_pairs, который строит пары со всеми элементами второго множества. Пары хранятся в виде структуры pair(X, Y). Для сцепления списков пар используется предикат append_pairs.

4. Примеры запросов к базе знаний и ее ответов



5. Листинг разработанной базы знаний на языке Пролог

```
domains
    element = integer
    set = element*
    pair = pair(element, element)
    pair_set = pair*

predicates
```

```
    nondeterm member(element, set)
    append(set, set, set)
    append_pairs(pair_set, pair_set, pair_set)
    union(set, set, set)
    nondeterm intersection(set, set, set)
    subtraction(set, set, set)
    cartesian_product(set, set, pair_set)
    add_unique(set, set, set, set)
    make_pairs(element, set, pair_set)
    write_set(set)
    write_pair_set(pair_set)
    nondeterm start

clauses
    member(X, [X | _]).
    member(X, [_ | T]) :- member(X, T).

    append([], L, L).
    append([H | T], L, [H | R]) :- append(T, L, R).

    append_pairs([], L, L).
    append_pairs([H | T], L, [H | R]) :- append_pairs(T, L, R).

    union(A, B, Result) :-
        append(A, [], Temp),
        add_unique(B, A, Temp, Result).

    add_unique([], _, Res, Res).
    add_unique([H | T], SetA, Current, Result) :-
        member(H, SetA),
        !,
        add_unique(T, SetA, Current, Result).
    add_unique([H | T], SetA, Current, Result) :-
        append(Current, [H], NewCurrent),
        add_unique(T, SetA, NewCurrent, Result).

    intersection(_, [], []).
    intersection([], _, []).
    intersection([H | T], B, [H | Res]) :-
        member(H, B),
        !,
        intersection(T, B, Res).
    intersection([_ | T], B, Res) :-
        intersection(T, B, Res).

    subtraction([], _, []).
    subtraction([H | T], B, Res) :-
        member(H, B),
        !,
        subtraction(T, B, Res).
    subtraction([H | T], B, [H | Res]) :-
        subtraction(T, B, Res).

    cartesian_product([], _, []).
    cartesian_product([H | T], B, Result) :-
        cartesian_product(T, B, Rest),
        make_pairs(H, B, Pairs),
        append_pairs(Pairs, Rest, Result).

    make_pairs(_, [], []).
    make_pairs(X, [Y | T], [pair(X, Y) | Rest]) :-
        make_pairs(X, T, Rest).

    write_set(S) :-
```

```
        write(S), nl.

    write_pair_set(PS) :-
        write(PS), nl.

    start :-
        A = [5, 7, 13, 14],
        B = [13, 14, 22],

        write("A = "), write_set(A),
        write("B = "), write_set(B), nl,

        union(A, B, U),
        write("Objedinenie: "), write_set(U),

        intersection(A, B, I),
        write("Peresechenie: "), write_set(I),

        subtraction(A, B, S1),
        write("Vychitanie (A\\B): "), write_set(S1),

        subtraction(B, A, S2),
        write("Vychitanie (B\\A): "), write_set(S2),

        cartesian_product(A, B, C),
        write("Dekartovo proizvedenie: "), write_pair_set(C), nl.

goal
    start.
```